IBM · MACHINE LEARNING CAPSTONE

Building a Course Recommender System

From exploratory data analysis to content-based and collaborative-filtering recommender systems — a complete machine learning journey.

Asmit Saha

AGENDA

What We'll Cover

01

Exploratory Data Analysis

Course titles, genres, enrollment behaviour and top courses.

02

Content-Based Recommenders

User profiles, course similarity, and clustering (unsupervised).

03

Collaborative Filtering

KNN, NMF, and Neural Network embeddings (supervised).

04

Conclusion

Key findings, model comparison, and final recommendation.

01

Exploratory Data Analysis

Understanding course titles, genres, and learner behaviour before building any model.

EXPLORATORY DATA ANALYSIS

Unveiling Popular IT Skills Through Course Titles

- A WordCloud visualization was generated across all online course titles to surface the most common keywords.
- Dominant themes: Python, Data Science, Machine Learning, Big Data, AI, TensorFlow, Containers, and Cloud Computing.
- Word size reflects frequency — larger words such as “data science” and “python” appear across the widest range of course titles.
- This gives educators and learners a quick read on which IT skills are most represented on the platform.

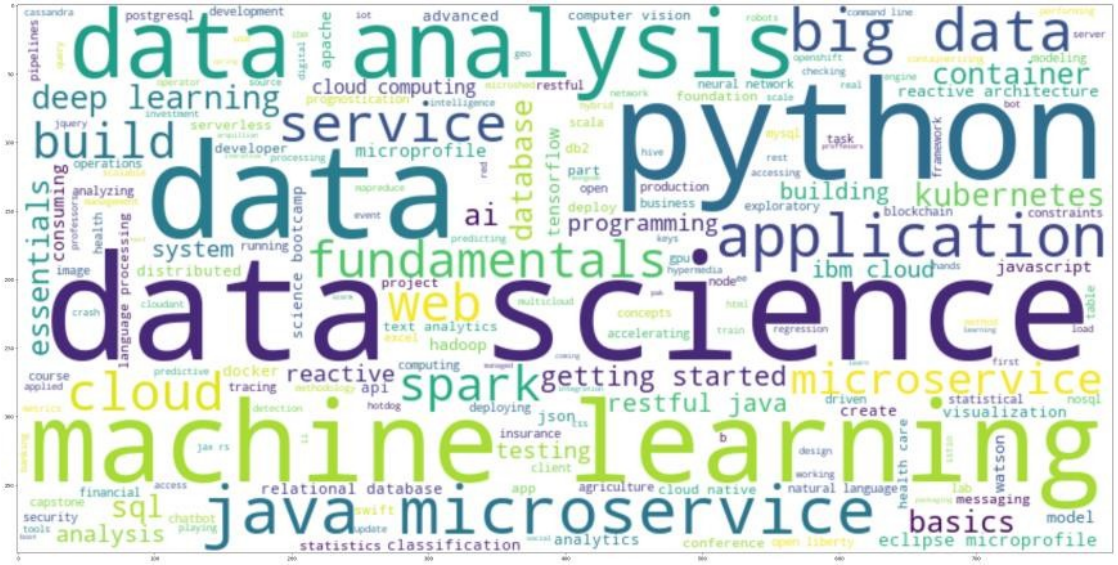


Figure: WordCloud of frequent keywords in course titles

8+
Recurring core IT skill themes identified

Python
Most frequently mentioned keyword

Data Science
Leading course subject area

EXPLORATORY DATA ANALYSIS

Course Genres: Popular Topics & Trends

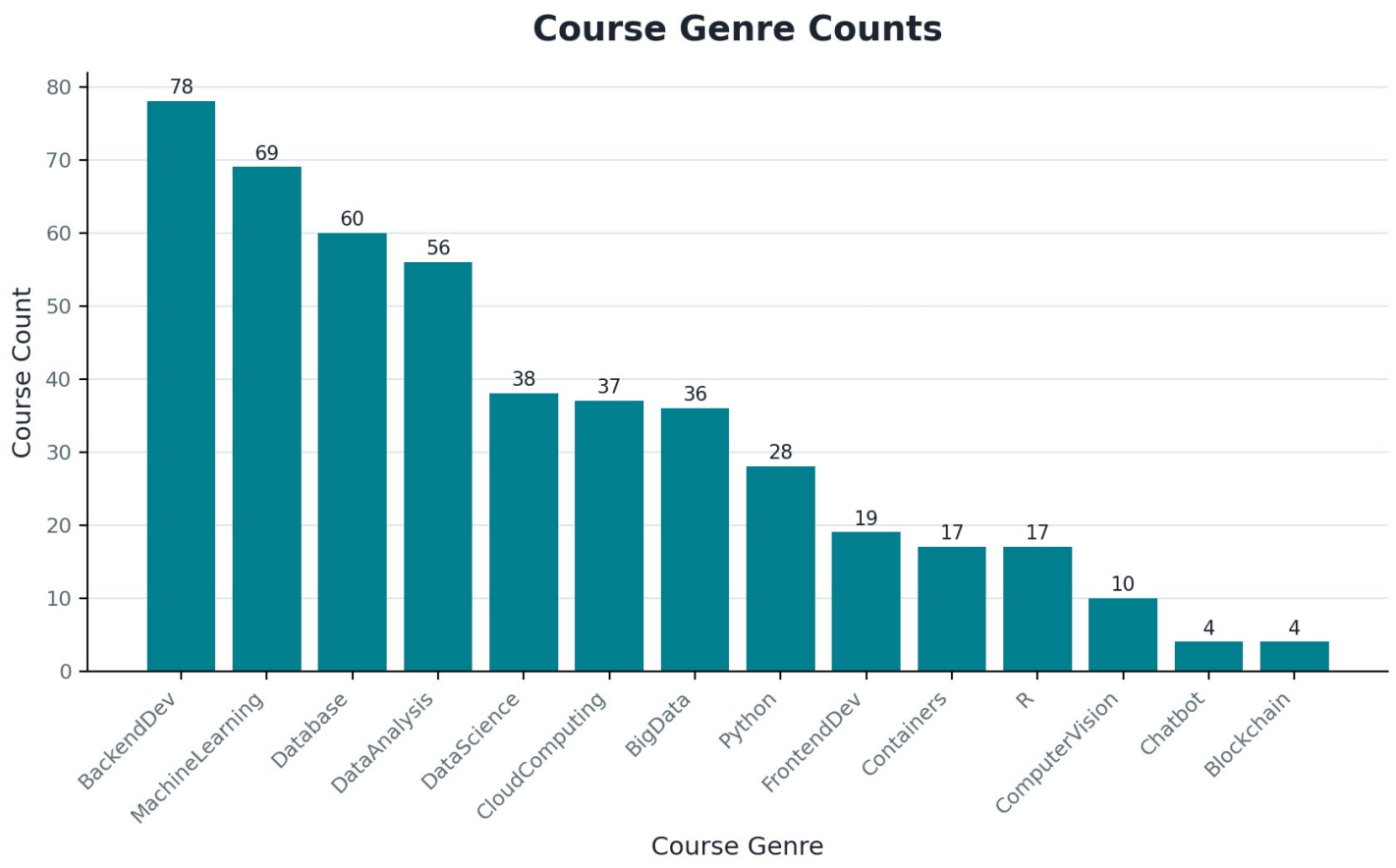


Figure: Bar chart of course counts by genre (14 genres)

- Backend Development (78), Machine Learning (69) and Database (60) are the most prevalent course genres.
- Data Analysis and Data Science follow closely, reinforcing the data-skills trend seen in the WordCloud.
- Blockchain and Chatbot courses are the least common — niche, emerging topics.
- This distribution guides which genres carry the most weight in user profile vectors.

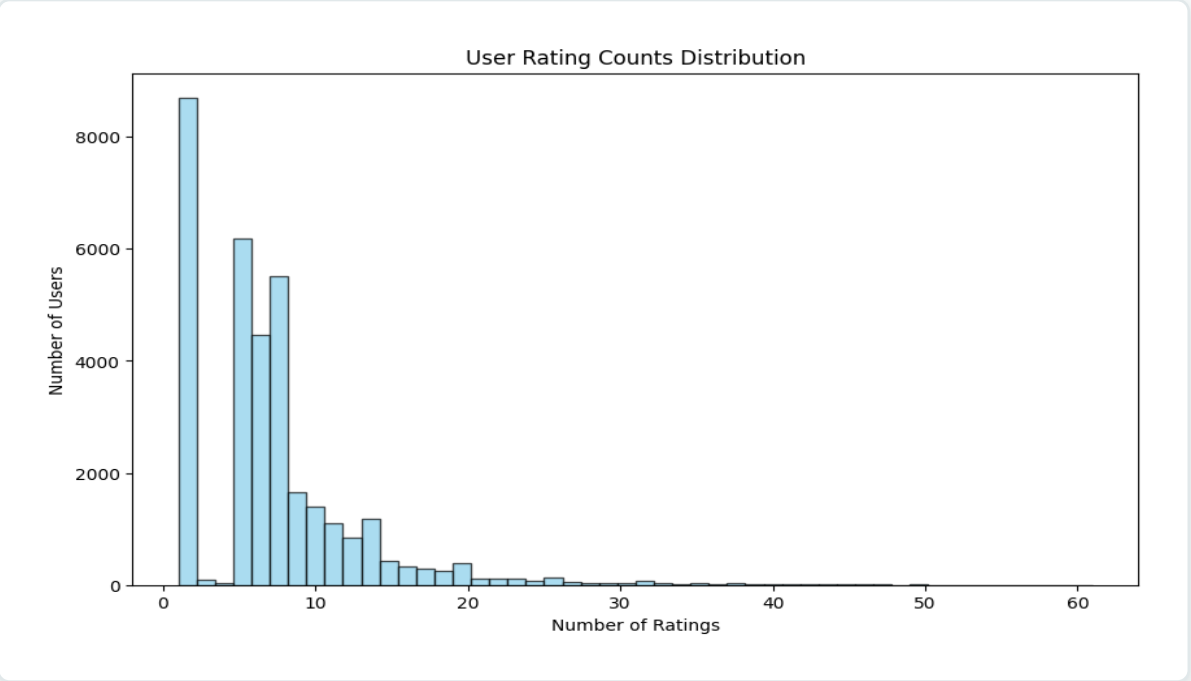
14

Distinct course genres analyzed

EXPLORATORY DATA ANALYSIS

Course Enrollments: User Engagement & Interactions

- Rating counts were aggregated per user to understand how actively learners engage with the platform.
- The dataset comprises 233,306 enrollment records from 5,000 unique users.
- Most users rate relatively few courses, while a smaller group is highly active — a classic long-tail distribution.
- This behavioural signal is exactly what powers collaborative-filtering models later in the project.



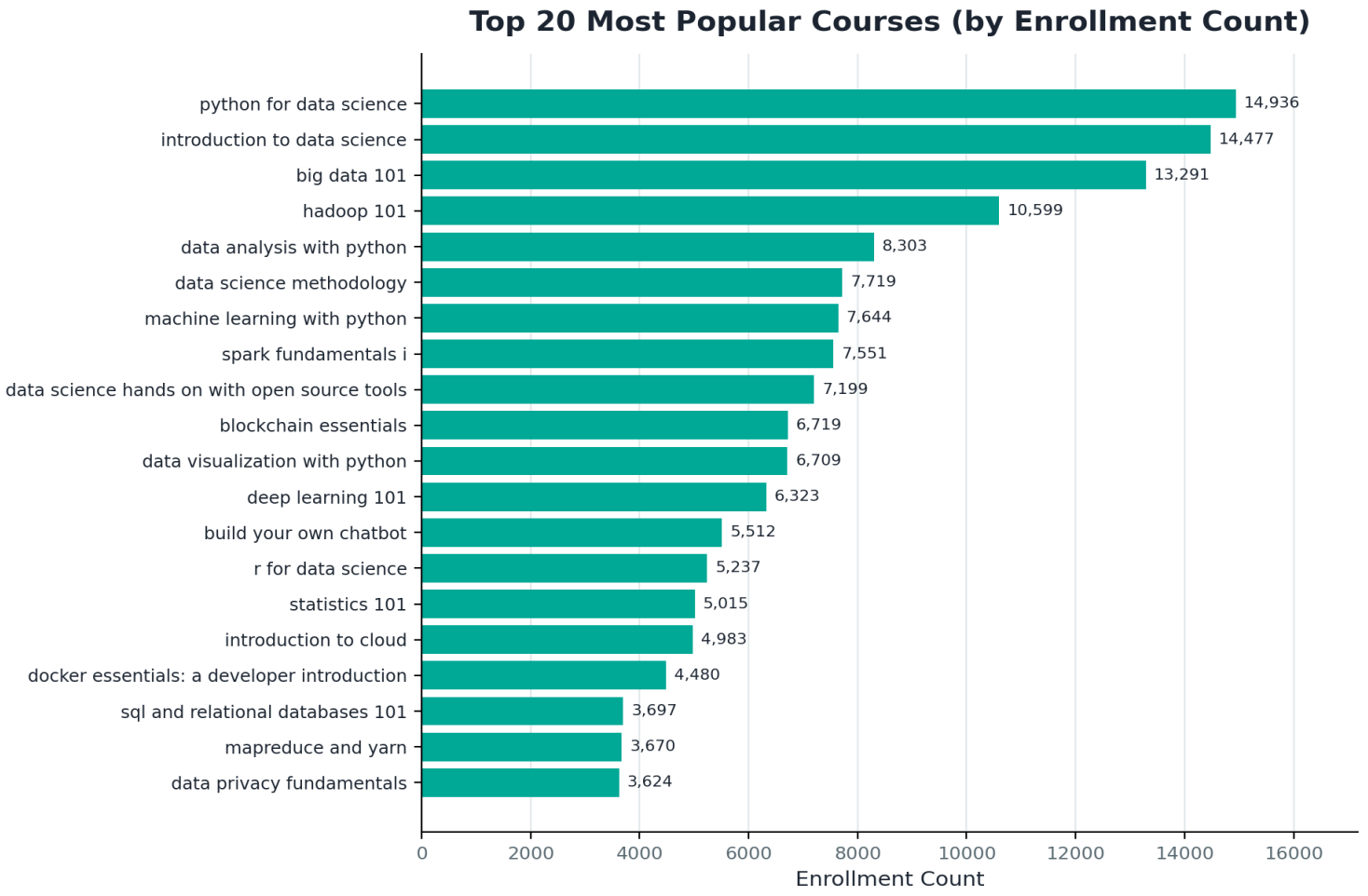
233,306
Total enrollment records

5,000
Unique users in the dataset

Long-tail
Shape of user engagement

EXPLORATORY DATA ANALYSIS

Identifying the Top 20 Most Popular Courses



- Courses aggregated and sorted by enrollment counts; the top 20 highlight learner preferences.
- “python for data science” and “introduction to data science” lead by a wide margin.
- Data & Python-related courses dominate the top of the list, echoing the WordCloud and genre analysis.

63.3%

of all enrollments come from just the top 20 courses

EXPLORATORY DATA ANALYSIS

Key Insights Before Modelling

1

Data-Skills Dominate

Python, Data Science & Machine Learning are the platform's core identity.

2

Concentrated Demand

63.3% of enrollments sit in only 20 courses — a strong signal for popularity-based baselines.

3

Long-Tail Engagement

Most users rate only a handful of courses, so recommenders must work well with sparse data.

4

14 Distinct Genres

Genre labels give a natural, interpretable feature set for content-based profiling.

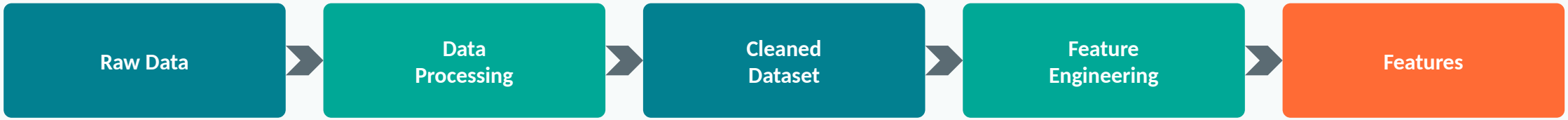
02

Content-Based Recommender Systems

Unsupervised approaches using user profiles, course similarity, and clustering.

CONTENT-BASED · UNSUPERVISED LEARNING

Recommender Using User Profile & Course Genres



- Raw data: user profiles, course genres, and course ratings/interactions.
- Data processing: handling missing values, removing duplicates, formatting for analysis.
- Cleaned dataset: consistent, error-free records ready for feature engineering.
- Feature engineering: build user profile vectors and course genre vectors that capture interests and course characteristics.

How Recommendations Are Generated

1. For each test user, take their user-profile vector (weights across the 14 course genres, built from courses they already rated).
2. Identify every course the user has NOT yet taken (“unknown” courses) and pull each one's course-genre vector.
3. Compute a recommendation score for each unknown course as the dot product of the user-profile vector and that course's genre vector.
4. Filter out any course scoring below the 10.0 threshold, then rank the rest and return the top-scoring courses as recommendations.

Evaluation: User Profile-Based Recommender

- Hyper-parameter: a recommendation score threshold of 10.0 filters out low-scoring, less-relevant recommendations.
- Coverage: on average, 61.82 new courses are recommended per user in the test set — the widest reach of all three content-based methods.
- This breadth comes from comparing full genre-vector profiles, which surfaces many plausible matches per learner.

10.0
Score threshold

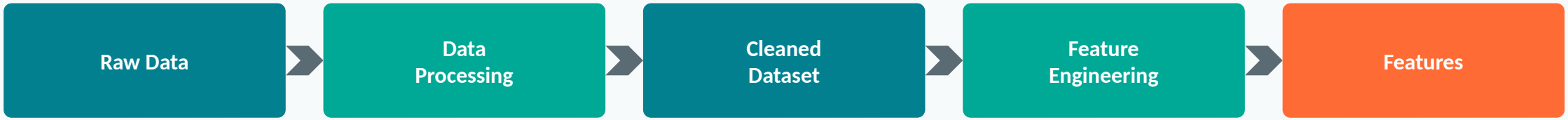
61.82
Avg. new courses / user

Top 10 Most Frequently Recommended Courses

Course ID	Recommendation Count
TA0106EN	608
GPXX0IBEN	548
excourse22	547
excourse21	547
ML0122EN	544
GPXX0TY1EN	533
excourse04	533
excourse06	533
excourse31	524
excourse73	516

CONTENT-BASED · UNSUPERVISED LEARNING

Recommender Using Course Similarity (TF-IDF)



- Raw data: course titles, descriptions, and other course attributes.
- Data processing: tokenization and lemmatization break text into base-form words.
- Cleaned dataset: stopwords and irrelevant/noisy outliers removed.
- Feature engineering: TF-IDF (Term Frequency-Inverse Document Frequency) vectors calculated per course.

How Recommendations Are Generated

1. Every course is represented as a TF-IDF vector built from its title and description text.
2. For a given course a user has already taken, compute the cosine similarity between its TF-IDF vector and every other course's vector.
3. Keep only course pairs with similarity ≥ 0.6 — this is the similarity threshold that defines “close enough to recommend.”
4. Rank the matching courses by similarity score and recommend the closest, most content-similar courses to the user.

Evaluation: Course Similarity-Based Recommender

- Hyper-parameter: a similarity threshold of 0.6 determines how closely two courses must match to be recommended.
- Coverage: only about 0.987 new courses are recommended per user on average — a far more conservative, precision-focused method.
- “excourse22” and “excourse62” are the most frequently recommended courses, each surfacing 257 times.

0.6
Similarity threshold

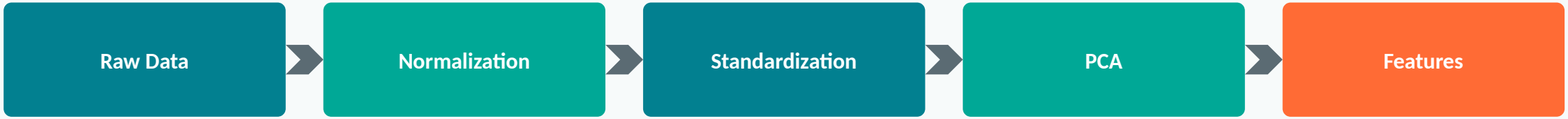
0.987
Avg. new courses / user

Top 10 Commonly Recommended Courses

Course ID	Times Recommended
excourse22	257
excourse62	257
WA0103EN	101
TA0105	41
DS0110EN	38
excourse46	24
excourse47	24
excourse63	23
excourse65	23
TMP0101EN	17

CONTENT-BASED · UNSUPERVISED LEARNING

Recommender Using User Profile Clustering



- Raw data: user profile feature vectors across course genres (Machine Learning, Data Science, Cloud Computing, etc.).
- Normalization: StandardScaler applied so all features share a similar scale and distribution.
- Standardization: each feature is transformed to mean 0 and standard deviation 1.
- PCA (Principal Component Analysis): reduces dimensionality while retaining the components that explain the most variance.

How Recommendations Are Generated

1. Each user's profile vector (genre weights) is standardized, then reduced with PCA to the components explaining >90% of variance.
2. K-means clusters users into groups with similar course-genre interests (cluster count chosen via the elbow method).
3. For a target user, find their cluster, then look at which courses other users in that same cluster have taken.
4. Recommend the courses most popular within that cluster that the target user hasn't taken yet.

Evaluation: Clustering-Based Recommender

- Hyper-parameters: cluster count chosen via the elbow method (K-means); PCA components chosen to explain over 90% of variance.
- Coverage: approximately 36.587 new courses recommended per user — a balanced middle ground between the other two methods.
- “WA0101EN,” “DB0101EN,” and “DS0301EN” are the top course recommendations, popular across multiple clusters.

Top 10 Most Frequently Recommended Courses

Course ID	Times Recommended
WA0101EN	864
DB0101EN	857
DS0301EN	856
CL0101EN	852
ST0101EN	800
CO0101EN	783
RP0101EN	773
CC0101EN	769
DB0151EN	741
ML0120EN	738

>90%
Variance explained (PCA)

36.587
Avg. new courses / user

CONTENT-BASED · COMPARISON

Comparing the Three Content-Based Approaches

Method	Hyper-parameter	Avg. Courses/User	Best suited for
User Profile + Genres	Score threshold = 10.0	61.82	Maximum coverage & discovery
Course Similarity (TF-IDF)	Similarity threshold = 0.6	0.987	High-precision, near-identical matches
User Profile Clustering	PCA > 90% var. + K-means	36.587	Balanced coverage with grouped personalization

- Innovative insight: coverage and precision trade off directly against each other — the user-profile method recommends 60x more courses per user than the course-similarity method, but at the cost of tighter relevance.
- Clustering sits in between, offering a practical default when the platform wants both discovery and relevance.
- In production, these methods could be blended: use course-similarity for “more like this,” and user-profile or clustering for “discover something new.”

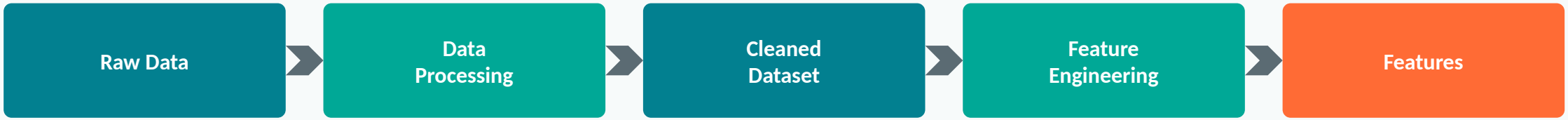
03

Collaborative Filtering

Supervised learning approaches: KNN, NMF, and Neural Network embeddings.

COLLABORATIVE FILTERING · SUPERVISED LEARNING

KNN-Based Recommender System



- Raw data: user-item pairs (user IDs, course IDs) with their ratings/enrollments.
- Data processing: loading, handling missing values, removing duplicates, formatting for modelling.
- Cleaned dataset: a structured, error-free basis for building the recommendation model.
- Feature engineering: the user-item ratings matrix is built, with each row a user's rating vector across all courses.

How Recommendations Are Generated

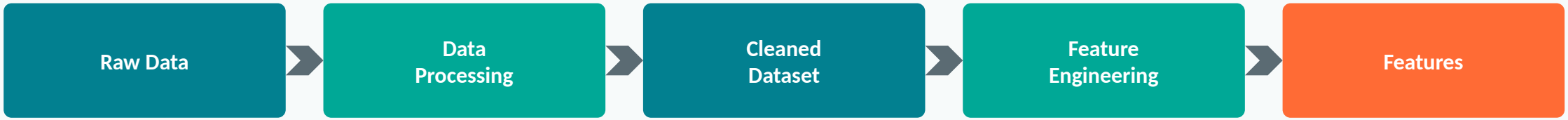
1. The KNN model represents each user (or item) as a vector of ratings and measures distance/similarity (e.g. cosine) between vectors.
2. For a target user-course pair, it finds the K most similar neighbouring users who have already rated that course.
3. It predicts the target user's rating as the (distance-weighted) average of those K neighbours' ratings for that course.
4. Courses with the highest predicted ratings that the user hasn't taken yet are surfaced as recommendations.

0.2063

Test RMSE (K-Nearest Neighbours)

COLLABORATIVE FILTERING · SUPERVISED LEARNING

NMF-Based Recommender System



- Raw data: course ratings data, unprocessed and possibly containing noise or missing values.
- Data processing: Pandas is used to pivot the data into a user-item matrix.
- Cleaned dataset: rows = users, columns = items (courses), cells = ratings/interactions.
- Feature engineering: latent factors are generated via Non-negative Matrix Factorization (NMF).

How Recommendations Are Generated

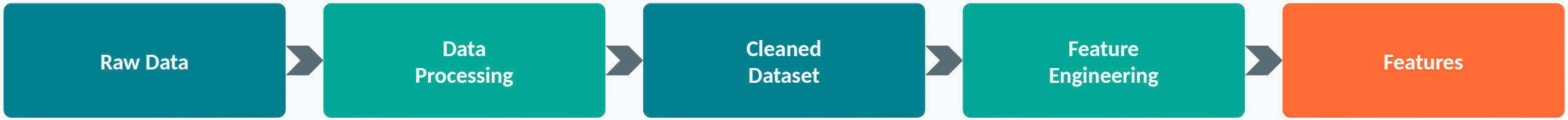
1. NMF factorizes the sparse user-item ratings matrix into two smaller matrices: a user-latent-factor matrix and an item-latent-factor matrix.
2. Each user and each course is represented by a short vector of latent factors (hidden themes learned directly from the ratings data).
3. A predicted rating for any user-course pair is reconstructed by taking the dot product of that user's and that course's latent vectors.
4. Unrated courses with the highest reconstructed/predicted ratings are recommended to the user.

0.2048

Test RMSE (Non-negative Matrix Factorization)

COLLABORATIVE FILTERING · SUPERVISED LEARNING

Neural Network Embedding-Based Recommender



- Raw data: the same course ratings / user-item interaction data used across all collaborative-filtering models.
- Data processing: pivoted into a user-item matrix, cleaned of duplicates and missing values.
- Cleaned dataset: structured user-item ratings ready for embedding-based modelling.
- Feature engineering: a neural network learns dense embeddings for both users and items.

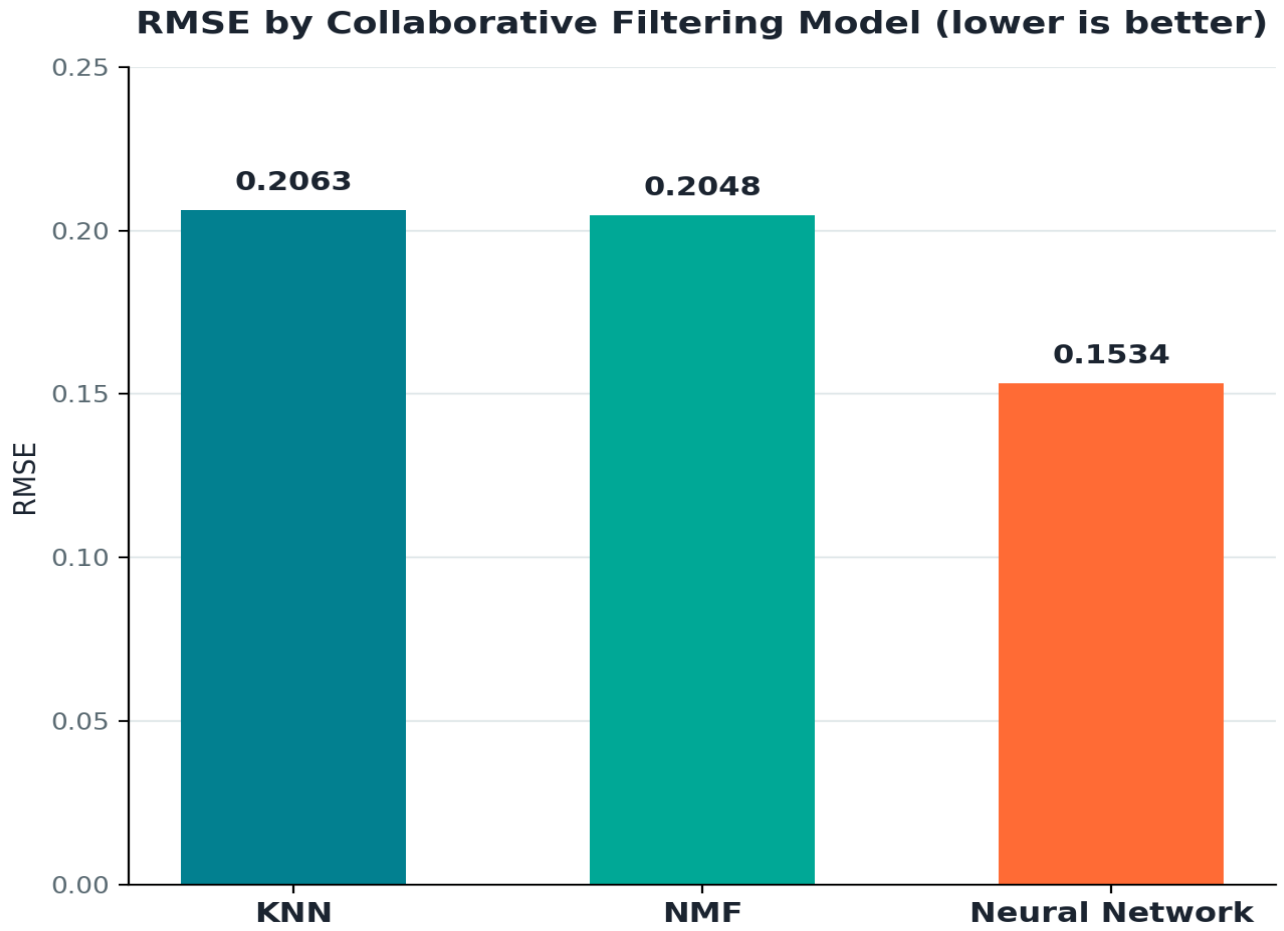
How Recommendations Are Generated

1. Each user and each course is mapped to a dense embedding vector, initialized randomly and learned during training.
2. The network combines a user's embedding and a course's embedding (e.g. dot product + dense layers) to predict a rating.
3. Training minimizes the error between predicted and actual ratings, adjusting embeddings so similar tastes/courses end up close together.
4. At inference, the model scores every course a user hasn't taken and recommends the highest-predicted ones.

0.1534

Test RMSE (Neural Network Embedding) — Best Result

Comparing Collaborative Filtering Models



- The Neural Network Embedding model achieved the lowest RMSE (0.1534), meaning the smallest average prediction error.
- Both KNN and NMF perform similarly (~0.205), as they rely on simpler linear/neighbourhood assumptions.
- The embedding model captures non-linear relationships between users and items, uncovering subtler preference patterns.

Neural Network

Best-performing collaborative filtering model

INNOVATIVE INSIGHT

Accuracy vs. Computational Cost

Model	RMSE	Relative Training Cost	Notes
KNN	0.2063	Low	Fast to set up, struggles with sparsity at scale
NMF	0.2048	Medium	Good latent-factor baseline, linear assumptions
Neural Network Embedding	0.1534	High	Best accuracy, needs more compute & training time

- The most accurate model is not always the cheapest to run — Neural Network embeddings need deeper architectures and longer training cycles.
- For a platform at scale, a practical strategy is to prototype quickly with KNN/NMF, then graduate to embeddings once the accuracy gain justifies the infrastructure cost.
- Recommendation: deploy the Neural Network Embedding model for the production recommender, given its clear accuracy advantage (25%+ lower RMSE than KNN).

04

Conclusion

Bringing together the exploratory findings and every recommender system built in this project.

CONCLUSION

What the Data and Content-Based Models Showed

Exploratory Data Analysis

- Backend Development, Machine Learning, and Database are the most popular course genres.
- 233,306 enrollments from 5,000 users show a long-tail engagement pattern.
- 63.3% of enrollments concentrate in the top 20 courses, led by Python and Data Science titles.
- These patterns confirm strong demand for data-related skills across the platform.

Content-Based Recommenders

- User-profile matching gives the broadest reach: 61.82 new courses recommended per user.
- Course-similarity (TF-IDF) is the most conservative — 0.987 new courses per user — but highly precise.
- Clustering with PCA + K-means balances the two, recommending 36.587 courses per user.
- Together, these methods show how the same course-genre data can power very different recommendation strategies.

CONCLUSION

Collaborative Filtering & Final Recommendation

- Three supervised collaborative-filtering models were built and evaluated on the same user-item ratings data: KNN, NMF, and Neural Network Embeddings.
- The Neural Network Embedding model achieved the lowest RMSE (0.1534), outperforming KNN (0.2063) and NMF (0.2048) by a clear margin.
- Its ability to learn non-linear relationships between users and courses makes it the most effective option for predicting ratings at scale.
- Final recommendation: combine a content-based method (for cold-start / new users) with the Neural Network Embedding model (for users with enough rating history) to deliver the strongest overall recommender system.

Final RMSE Scoreboard

KNN	0.2063
NMF	0.2048
Neural Network	0.1534

Thank You

Questions & Discussion

Asmit Saha · IBM Machine Learning Capstone